

Stylometric Detection of AI-Generated Phishing

Distinguishing Known AI-Generated Credential-Harvesting Emails from Real-World Phishing

Author: Agboola Michael Daramola

Supervisor: Dr Radu Negoescu

Module: CO4011 Master's Project (Level 7)

Programme: MSc Cybersecurity with AI

Institution: University of Lancashire

Deliverable 2 - Project Report

Abstract

Large language models have removed the historical trade-off between the scale and the quality of phishing, allowing attackers to mass-produce fluent, personalised credential-harvesting emails that evade the spelling and grammar cues on which many defences once relied. Building on Eze and Shamir (2024), who showed that stylistic text analysis can separate AI-generated phishing from human email classes, this project is a focused replication and extension that asks a single question within one narrow attack type: to what extent do interpretable stylometric features separate known AI-generated phishing from real-world phishing, and which markers carry the signal? A balanced corpus of 982 credential-harvesting emails was assembled from two published research sources - 491 real-world phishing messages from the Nazario corpus (2022–2024) and 491 known AI-generated messages from the Eze and Shamir (2024) dataset - after a documented, rule-based credential-harvesting screen. Each email body was reduced to a vector of 95 interpretable stylometric and lexical features, computed with no access to the generating model. Logistic regression, Random Forest and XGBoost classifiers were trained through a fully seeded, leakage-controlled pipeline and evaluated on a held-out test set excluded from model fitting and hyperparameter selection. All three models exceeded the $F1 = 0.90$ target pre-specified in the approved proposal: logistic regression achieved $F1 = 0.987$ (95% CI [0.965, 1.000]) and ROC-AUC = 0.995, XGBoost achieved $F1 = 0.980$, and Random Forest achieved $F1 = 0.973$. Class-level means show a pronounced readability and word-length difference, while model-based importance analyses indicate that both readability/length features and function-word patterns contribute; their exact ordering depends on the importance method. Removing URL and email-address counts changed performance little. Performance also remained high on a similarity-cluster grouped partition, although that run used more training data than the random split and is therefore corroborative rather than a controlled comparison. A manual audit of the corpus screen is reported, including a source-asymmetry defect it uncovered and corrected; because the same audit sample informed the correction and the final scoring, the screening estimates are exploratory. The study concludes that stylometry is a viable and interpretable detection signal for this source pairing, subject to the limitations that the AI class comes from a single published generation dataset, authorship of the real-world class is not verified, broader source confounding remains possible, and deliberate paraphrasing may weaken detection.

Table of Contents

ABSTRACT	II
LIST OF FIGURES	V
LIST OF TABLES.....	VI
1. INTRODUCTION	1
2. STATE OF THE ART	4
2.1 THE THREAT: EFFECTIVENESS OF LLM-GENERATED PHISHING	4
2.2 DETECTING MACHINE-GENERATED TEXT IN GENERAL	4
2.3 STYLOMETRY AND AUTHORSHIP ATTRIBUTION	5
2.4 DETECTING AI-GENERATED PHISHING SPECIFICALLY	6
2.5 CLASSICAL PHISHING DETECTION AND ITS BLIND SPOT	6
2.6 RESEARCH METHODOLOGIES IN THE FIELD AND THEIR IMPLICATIONS.....	7
2.7 THE GAP THIS PROJECT ADDRESSES.....	7
3. METHODOLOGY	8
3.1 RESEARCH DESIGN.....	8
3.2 DATA COLLECTION AND SAMPLING.....	8
3.3 ANALYSIS METHODS.....	8
3.4 OBJECTIVE COMPLETION CRITERIA	9
3.5 EVALUATION PROCESS	9
3.6 ETHICAL CONSIDERATIONS	10
3.7 LIMITATIONS.....	10
4. DESIGN	11
4.1 REQUIREMENTS ANALYSIS	11
4.2 SYSTEM ARCHITECTURE	11
4.3 FEATURE-SET SPECIFICATION	12
4.4 REQUIREMENTS PRIORITISATION (MoSCoW)	13
5. IMPLEMENTATION	14
5.1 CORPUS CONSTRUCTION AND CLASS COMPARABILITY	14
5.2 VALIDATING THE CREDENTIAL-HARVESTING SCREEN	15
5.3 PREVENTING TRAIN/TEST LEAKAGE	15
5.4 FEATURE-EXTRACTION EDGE CASES.....	16
5.5 HYPERPARAMETER SEARCH UNDER MODEST COMPUTE.....	16
5.6 CUTTING THE TRANSFORMER BASELINE.....	17
5.7 TESTING AND SANITY BASELINES.....	17

6. EVALUATION.....	18
6.1 HEADLINE RESULTS.....	18
6.2 CONFUSION MATRICES.....	18
6.3 WHICH STYLISTIC MARKERS REVEAL MACHINE AUTHORSHIP.....	19
6.4 STRUCTURAL-CUE ABLATION.....	23
6.5 SIMILARITY-CLUSTER GROUPED SPLIT AND 2022-ONLY SENSITIVITY.....	23
6.6 REVISITING THE AIM, AND WHAT THE EVALUATION CANNOT SHOW.....	25
7. DISCUSSION.....	26
7.1 DOES WRITING STYLE BETRAY MACHINE AUTHORSHIP?.....	26
7.2 RELATIONSHIP TO THE LITERATURE.....	26
7.3 DOES THE ARTEFACT SOLVE THE STATED PROBLEM?.....	26
8. CONCLUSION.....	28
REFERENCES.....	29
APPENDIX A: ARTEFACT AND SOURCE CODE.....	32
APPENDIX B: CORPUS CONSTRUCTION AND PROVENANCE.....	32
APPENDIX C: SOURCE CODE AND PSEUDO-CODE.....	34
C.1 PSEUDO-CODE: THE CREDENTIAL-HARVESTING SCREEN.....	34
C.2 PSEUDO-CODE: CAMPAIGN-LEVEL GROUPED PARTITION.....	34
C.3 FEATURE EXTRACTION (SRC/FEATURES.PY, EXTRACT).....	35
C.4 CREDENTIAL-HARVESTING SCREEN (SCRIPTS/BUILD_CORPUS.PY, EXTRACT).....	35
C.5 BOOTSTRAP CONFIDENCE INTERVAL (SRC/METRICS.PY, EXTRACT).....	35

List of Figures

Figure 4.1 - Pipeline architecture. Each stage has a single responsibility and a fixed interface, and every stochastic step is seeded for reproducibility.

Figure 6.1 - Confusion matrix for the logistic-regression model on the held-out test set. Rows are the true class, columns the prediction.

Figure 6.2 - Random-Forest impurity importance (top 20 features).

Figure 6.3 - Distribution of Flesch Reading Ease by class. The real-world distribution sits markedly higher (easier to read); dashed lines mark the class means.

List of Tables

Table 4.1 - The 95-feature stylometric and lexical feature set (see `src/features.py` for exact definitions).

Table 6.1 - Random-split test-set performance for the positive (AI-generated) class. Interpret alongside the similarity-cluster split in Table 6.4.

Table 6.2 - Direction of the strongest signals: per-class feature means over the full corpus.

Table 6.3 - Top eight features by each importance method. The two methods share only 7 features across their full top-20 lists (Spearman $\rho = 0.00$), so both are reported rather than combined.

Table 6.4 - F1 with 95% bootstrap intervals under robustness and sensitivity conditions, for all three classifiers. The signal persists under similarity-cluster splitting and on the 2022-only sensitivity subset.

Table B.1 - Corpus construction and screening funnel; every stage is reported so the counts reconcile to the final 982-message corpus.

Table B.2 - Corpus parameters, source provenance and exploratory screen-audit performance.

Table B.3 - Provenance checksums of the raw source files (full hashes in `data/corpus_summary.json`).

1. Introduction

Phishing remains the single most common route by which attackers gain an initial foothold in an organisation, and the harvesting of user credentials is its dominant objective. For most of its history the economics of phishing imposed a natural ceiling on quality: a convincing, well-targeted message required human effort, so an attacker had to trade scale against persuasiveness. The maturation of large language models (LLMs) has dissolved that trade-off. A capable model can now be instructed to produce fluent, grammatically clean and context-aware credential-harvesting emails in seconds, and to emit a fresh variant for every recipient. Heiding et al. (2024) found that fully automated GPT-4 phishing outperformed generic control messages (roughly 30–44% click-through) but did not, on its own, match emails designed with the human expert V-Triad framework (roughly 69–79%); a hybrid of GPT-4 and the V-Triad performed comparably to, and in some conditions better than, the human-only method - all at a fraction of the human effort. The threat is therefore one of scale and cost-efficiency as much as raw persuasiveness. The problem this project addresses is the detection of machine-authored phishing, and specifically whether the writing style of such messages betrays their non-human origin.

The problem decomposes into three connected parts. First, the industrialisation of phishing by LLMs raises both the volume and the average quality of attacks, eroding the grammatical and spelling errors on which users and some filters have traditionally relied. Second, because a generative model can produce a unique message for each victim, defences that match known-bad signatures or blocklisted content lose much of their purchase (Eze and Shamir, 2024). Third, and least understood, is the question of authorship: it is not yet established whether the stylistic regularities of machine-generated prose are strong and stable enough to act as a reliable detection signal for phishing specifically, rather than for long-form text in general. This project isolates and investigates that third component.

Several standards and legal instruments frame the work. Email corpora may contain personal data - names, addresses and other identifiers - so the project treats its source data as personal data under the UK General Data Protection Regulation and the Data Protection Act 2018 unless effective anonymisation can be demonstrated, and the principles of data minimisation and storage limitation govern how it is handled. The Computer Misuse Act 1990 makes the boundary between defensive research and offensive capability legally significant, and this project sits firmly on the defensive side: it detects, and never deploys, phishing. More broadly the work aligns with the risk-management framing of ISO/IEC 27001 (International Organization for Standardization, 2022) and with the anti-

phishing guidance issued by national bodies such as the UK National Cyber Security Centre. These considerations are developed in the Methodology.

The significance of the problem follows from its trajectory. Industry telemetry such as the Anti-Phishing Working Group's trends reports and the annual Verizon Data Breach Investigations Report consistently place phishing and stolen credentials among the leading causes of security breaches (APWG, 2025; Verizon, 2025). If the problem is solved, even partially, a stylometric detector could add a lightweight, content-based layer to existing mail-filtering and security-operations tooling, flagging messages whose style is characteristic of machine generation. If it is not, defenders face an adversary that can scale high-quality, personalised social engineering almost without limit while the cues that once exposed phishing steadily vanish. The asymmetry of these outcomes motivates a focused, rigorous study.

This project works towards a solution through a controlled, reproducible machine-learning experiment. It deliberately restricts its scope to one attack type (credential-harvesting emails), one family of features (stylometric and lexical), and one binary decision (known AI-generated versus real-world phishing), so that the outcome is a definitive answer to a single question rather than a broad but shallow survey.

Aim. To determine the extent to which stylometric and lexical features enable machine-learning classifiers to distinguish known AI-generated credential-harvesting phishing emails from real-world (Nazario) phishing emails, and to identify which features are most discriminative.

Objectives.

1. Critically review the literature on AI-generated phishing, machine-generated-text detection and stylometric authorship attribution, establishing the boundary between what is known and what is not.
2. Assemble a balanced, fully documented binary corpus of real-world and known AI-generated credential-harvesting emails from published research sources.
3. Specify and implement a stylometric and lexical feature set that is interpretable as well as discriminative.
4. Train and tune Random Forest and XGBoost classifiers, with a standardised logistic-regression baseline, using a reproducible, seed-fixed pipeline.
5. Evaluate all models on a held-out test set using precision, recall and F1 with bootstrap confidence intervals, against a target F1 of at least 0.90.
6. Rank and interpret feature importance to identify the stylistic markers that most reveal machine authorship.

7. Test whether performance depends on selected corpus-specific structural cues, and assess robustness to leakage between detected textual-template clusters.

2. State of the Art

2.1 The threat: effectiveness of LLM-generated phishing

The starting point for this review is evidence that machine-generated phishing is a real and potent threat rather than a speculative one. Heiding et al. (2024) compared phishing produced automatically by GPT-4 with messages designed using the human expert V-Triad framework. Fully automated GPT-4 emails beat generic controls but did not, alone, reach the click-through rates of the human-designed messages; it was the hybrid of GPT-4 and the V-Triad that matched or exceeded the human-only method. Their results matter here in two ways: they establish the motivation - cheap, scalable phishing that is already competitive once lightly steered by a human - and they show that fluency and correctness, the properties LLMs excel at, are central to the threat. This suggests, in turn, that any residual stylistic signature of machine generation is where a defender should look.

2.2 Detecting machine-generated text in general

A large and fast-moving body of work addresses the detection of AI-generated text irrespective of domain, prompted by the step change in fluency that followed large-scale autoregressive models (Brown et al., 2020). Three families of method have emerged: supervised classifiers that learn to separate human from machine text from labelled examples; zero-shot statistical methods that exploit intrinsic properties of model output; and watermarking, which embeds a detectable signal at generation time. The most influential zero-shot method, DetectGPT (Mitchell et al., 2023), observes that machine-generated passages occupy regions of negative curvature in a model's log-probability surface, detecting them by measuring how sharply likelihood changes under small perturbations; it improved on earlier perplexity baselines markedly. This family is fragile, however: Krishna et al. (2023) showed that paraphrasing collapses the accuracy of DetectGPT, watermarking and commercial detectors alike - DetectGPT's detection rate fell from roughly 70% to under 5% at a fixed false-positive rate - while meaning is preserved. Two implications follow for the present work: detectors that need the generating model's probabilities are impractical for an email defender who does not control the attacker's model, and robustness must be treated as an explicit limitation rather than assumed away.

The supervised family is the closest cousin of this project's approach. Solaiman et al. (2019), reporting on the staged release of GPT-2, found that a fine-tuned classifier detected that model's output accurately but degraded as the generator grew more capable and sampling strategies changed - an early warning that detectability is tied to a specific generator. Zellers et al. (2019), with

Grover, argued more sharply that the strongest detector of neural text is often a model from the same family as the generator, precisely the assumption an email defender cannot make. Tools such as GLTR (Gehrmann et al., 2019) exposed the per-token fingerprints of machine text to human reviewers, confirming the signal is real but generator-dependent. This strand motivates the design choice at the heart of the present work: rather than depend on the generator's probabilities or a same-family reference model, it uses stylometric features a defender can compute from message text alone, trading theoretical power for practicality and interpretability.

Watermarking addresses a different point in the pipeline. Kirchenbauer et al. (2023) proposed biasing token sampling towards a secret 'green list' at generation time, so output carries a statistically detectable signature invisible to readers. This is elegant where the defender controls the generator, but inapplicable to phishing, where the attacker simply uses an unwatermarked model; the paraphrasing attack that breaks post-hoc detectors also weakens watermarks (Krishna et al., 2023). The literature therefore describes an arms race in which every signal has a corresponding evasion, and in which the defender's realistic assumptions - no model access, no generator cooperation - rule out the most powerful methods. That constraint, rather than preference, is what makes a transparent stylometric approach worth investigating for this bounded problem.

2.3 Stylometry and authorship attribution

The idea that writing style carries a measurable, author-specific signature predates modern machine learning by decades. The seminal study of Mosteller and Wallace (1964) attributed the disputed Federalist Papers by analysing the frequencies of common function words - articles, prepositions, conjunctions - on the grounds that such words are used unconsciously and are largely independent of topic. Function-word frequencies have since become a standard feature in authorship attribution precisely because of this topic-independence and resistance to deliberate manipulation (Stamatatos, 2009). Surveys of the field by Juola (2006) and Koppel, Schler and Argamon (2009) confirm that such unconscious markers, rather than topical content, carry most of the attributive signal. The relevance here is direct: if human and machine authors differ in their unconscious stylistic habits - sentence-length regularity, vocabulary richness, punctuation and function-word distributions - then a stylometric feature set offers an interpretable, generator-access-independent alternative to probability-based detectors. This project transfers the authorship-attribution toolkit from its classical human-versus-human setting to the human-versus-machine problem within the narrow domain of phishing email.

2.4 Detecting AI-generated phishing specifically

A smaller but growing literature targets AI-generated phishing directly, and Eze and Shamir (2024) are the direct precedent for this work - they supply its AI class and its method. They analysed a corpus of AI-generated phishing with UDAT, a stylistic approach built on characteristics such as punctuation, word length, readability, parts of speech and repetition, ranked the most informative descriptors (reporting differences in word length, pronoun and verb frequency, lexical diversity and sentence length), and reported two-way classification between AI-generated and human scam email with high accuracy. Koide et al. (2024), with ChatSpamDetector, took a complementary route, prompting an LLM to classify emails and explain its decisions. Together these studies establish that stylistic detection of AI-generated phishing is tractable and that several stylistic descriptors are informative. This project is therefore not the first demonstration of style-based detection; it is a focused replication and extension.

The extension is specific. Eze and Shamir evaluate over a mix of email types and against legitimate and human scam email, so a classifier can lean on topic or structural signals as well as style, and the reported accuracy is treated as the endpoint. The present work holds several variables fixed instead: it compares a single attack type (credential harvesting) against real-world phishing rather than legitimate mail, restricts the evidence to body-text style through a transparent 95-feature implementation, adds seed-fixed evaluation with bootstrap confidence intervals, ranks the responsible features by two methods, and adds an explicit ablation of structural cues plus a similarity-cluster grouped split to probe whether the accuracy is an artefact of corpus construction. The contribution is thus a narrower, better-characterised and more reproducible account of a signal earlier work already showed exists.

2.5 Classical phishing detection and its blind spot

Conventional phishing detection has relied on features drawn from message headers, embedded URLs and sender reputation, alongside content-based natural-language features. The canonical work, Fette, Sadeh and Tomasic (2007), trained a classifier ('PILFER') on ten hand-engineered structural features - link counts, IP-based URLs, mismatched href domains and similar signals - reporting detection well above 90%, and Abu-Nimeh et al. (2007) compared six machine-learning methods on a comparable feature set, establishing the experimental template the field still follows. Later work broadened these feature sets, from the statistical topic and link models of Bergholz et al. (2010) to URL-centric classifiers (Sahingoz et al., 2019), and Das et al. (2020) survey the resulting literature systematically. These methods remain valuable but predate the generative-AI threat, and their most powerful features are artefacts of the message envelope rather than properties of its

prose - the very features an LLM-generated message may lack or randomise. More fundamentally they answer 'is this message malicious?' rather than 'was this message written by a machine?'. The authorship question is orthogonal to, and complementary with, the maliciousness question, and it is this gap the project occupies while deliberately avoiding envelope artefacts (Section 6.4).

2.6 Research methodologies in the field and their implications

Methodologically, the reviewed work is dominated by quantitative, experimental studies: authors construct or obtain a labelled corpus, engineer or learn features, train and tune classifiers, and report standard classification metrics on held-out data. Recurring weaknesses are visible across the field - modest or single-source datasets, reliance on a single generator model, thin reporting of statistical uncertainty, and incomplete reproducibility. These observations directly shaped the methodology adopted here: a quantitative experimental design is appropriate and well-precedented, but it must be strengthened with explicit reproducibility guarantees, confidence intervals on the headline metric, and an honest, up-front statement of the single-generator limitation.

2.7 The gap this project addresses

In summary, it is known that LLM-generated phishing is effective when combined with human design (Heiding et al., 2024), that machine text is in principle detectable but that probability-based detectors are fragile under paraphrasing and require model access (Mitchell et al., 2023; Krishna et al., 2023), that stylometry can attribute authorship from unconscious stylistic habits (Mosteller and Wallace, 1964; Stamatatos, 2009), and - most directly - that stylistic analysis can already separate AI-generated phishing from human email and identify informative descriptors (Eze and Shamir, 2024; Koide et al., 2024). What remains less well characterised is how that signal behaves under tighter controls: within a single attack type, against real-world phishing rather than legitimate mail, with a transparent feature set whose contributions are ranked, and with explicit tests of whether the performance survives structural-cue ablation and similarity-cluster leakage control. Providing that narrower, reproducible characterisation - an extension of Eze and Shamir rather than a first demonstration - is the contribution of this project.

3. Methodology

3.1 Research design

The project adopts a quantitative, experimental research design. The research question is fundamentally measurable - it asks to what extent a classifier can separate two classes and which features drive that separation - so a quantitative approach is more appropriate than a qualitative one, and it is consistent with the dominant methodology in the field (Section 2.6). The design is comparative: several models are trained on a common feature representation and evaluated under identical conditions, so that differences in performance can be attributed to the models and features rather than to inconsistent experimental treatment.

3.2 Data collection and sampling

The study uses secondary data only, which keeps it reproducible and avoids the ethical hazard of generating fresh attack content. Each class is named for what its source actually guarantees. The first is real-world phishing from the Nazario corpus; its 2022–2024 archives were chosen so the class is broadly contemporaneous with the AI class, reducing an era-of-writing confound. It is treated as human-origin, but the corpus is hand-classified as phishing rather than verified for authorship, so it is labelled 'real-world' rather than 'human-written' (Section 3.7). The second is known AI-generated phishing from the published Eze and Shamir (2024) dataset, used rather than produced afresh. Both sources are reduced to email body text so the classifier sees the same kind of evidence for each class, and a rule-based credential-harvesting screen retains only messages pairing an account- or credential-related subject with an explicit action request; Section 5.1 defines the rule and reports the exclusions per source. The classes are then balanced by construction, to prevent the classifier exploiting prior class frequency, and de-duplicated before any splitting. A stratified 70/15/15 split is produced under seed 42, with hyperparameters selected by five-fold cross-validation inside the training partition only. Section 5.3 gives the exact partition counts and the role of each partition.

3.3 Analysis methods

Each email body is converted into a fixed-length vector of interpretable stylometric and lexical features (specified in Chapter 4): measures of length, vocabulary richness, readability, punctuation and orthography, structural cues, and function-word frequencies. Logistic regression, Random Forest and XGBoost classifiers are trained on these vectors, the first as a standardised baseline floor and the latter two as the primary models. Hyperparameters are selected by five-fold cross-validated grid search on the training data only. Performance is analysed with precision, recall and F1 for the

positive (AI-generated) class, supplemented by ROC-AUC and confusion matrices, and the uncertainty of the headline F1 is quantified with a percentile bootstrap confidence interval. Feature importance is analysed with both impurity-based and permutation-based methods so the interpretation does not rest on a single, potentially biased estimator, and per-class feature means are inspected to establish the direction of each signal.

The choice of metric matters because it is where several reviewed studies were weakest. Accuracy alone hides the trade-off between the two error types that matter to a defender - a missed machine-phishing email versus a wrongly flagged real-world one - so precision, recall and their harmonic mean, F1, are reported for the positive class, with ROC-AUC across all decision thresholds. Because a single point estimate can mislead, the headline F1 carries a percentile bootstrap 95% confidence interval computed by resampling the test predictions, guarding against over-claiming from a lucky split. A success threshold of $F1 = 0.90$, pre-specified in the approved proposal, fixes the bar before the results are seen so that the evaluation cannot be retrofitted to whatever the models happened to achieve. (This is a proposal-stated target, not a formal public preregistration.)

3.4 Objective completion criteria

Each objective has an explicit test of completion. Objective 1 is complete when the literature review establishes the known/unknown boundary with adequate coverage; Objective 2 when a balanced, de-duplicated, documented corpus exists; Objective 3 when the feature extractor produces a stable vector and passes its unit tests; Objective 4 when the models are trained with tuned hyperparameters and a fixed seed; Objective 5 when all models are evaluated on the held-out test set excluded from fitting and tuning with confidence intervals; Objective 6 when a ranked, interpreted list of discriminative features has been produced; and Objective 7 when structural-feature ablation and grouped splitting have quantified the extent to which selected corpus cues and detected template clusters influence performance. This progression is staged so that progress is observable and reportable at each step.

3.5 Evaluation process

Evaluation proceeds in two layers. The first is quantitative and answers whether the aim has been met: does the best model reach the target F1 of 0.90 on held-out data, and how does it compare with the simple baselines and with published results? The second is interpretive and answers the second half of the aim: which stylistic markers most reveal machine authorship, and are they consistent with the theoretical expectation that machine prose is more fluent and regular? Critically, the evaluation also reflects on its own limits - in particular, what the chosen metrics cannot reveal about generalisation to generator models not seen during training.

3.6 Ethical considerations

The work is defensive dual-use research and is treated accordingly. No message is ever sent to any recipient, no real individual or organisation is targeted, and no new phishing content is generated for release. Email corpora may contain personal data, so under UK GDPR and the Data Protection Act 2018 the project treats the source data as personal data unless effective anonymisation can be demonstrated. It therefore uses pre-existing public research corpora, collects no additional personal data, stores data on an encrypted device, publishes only aggregate statistics and derived features, and deletes the working data at project end per university policy. The Nazario corpus is used under its CC BY 4.0 licence with attribution to Jose Nazario. No human participants are involved, so no participant information sheet or consent form is required; on that basis the module's supervisor-led ethics process confirmed the project before any data handling began, and no separate formal approval number was issued for this secondary-data study.

3.7 Limitations

Four limitations are acknowledged in advance. First, the AI class originates from a single published AI-generation dataset produced through the DeepAI text-generation service, so the findings speak to detecting that dataset's output; the exact underlying model and version are not established by the source, and generalisation to other generators is untested. Second, the real-world (Nazario) class is hand-classified as phishing but not verified as human-authored, and because its archives overlap the period of public LLM availability, some AI-assisted contamination cannot be ruled out; a 2022-only sensitivity analysis (Section 6.5) partially addresses this. Third, the study is limited to English-language credential-harvesting email. Fourth, consistent with Krishna et al. (2023), a stylometric detector may be vulnerable to deliberate paraphrasing or 'humanisation' of machine output - an avenue for future work rather than a defect of the present study. These limitations are revisited against the actual results in Chapters 6 and 7.

4. Design

4.1 Requirements analysis

The functional requirements of the artefact follow directly from the objectives. The system must (F1) ingest a labelled corpus of emails and clean and de-duplicate it; (F2) extract a fixed, interpretable stylometric and lexical feature vector from each message; (F3) train and tune the baseline, Random Forest and XGBoost models; (F4) evaluate every model on a held-out test set and report precision, recall, F1, ROC-AUC and confidence intervals; and (F5) produce a ranked, interpreted feature-importance analysis. The non-functional requirements matter just as much for a research artefact: the pipeline must be reproducible (every stochastic step seeded), transparent (features must be human-interpretable so results can be explained and defended in the viva), efficient enough to run on a laptop, and testable (feature extraction and metrics covered by unit tests).

Interpretability was elevated from a nicety to a hard requirement, and this shaped the whole design. A more powerful detector could in principle be built by fine-tuning a transformer such as BERT (Devlin et al., 2019) directly on the raw text, and the literature confirms such models are strong text classifiers. But a transformer's decision is opaque, and the second half of the aim - identifying which stylistic markers reveal machine authorship - cannot be answered by a model whose features are uninterpretable latent dimensions. Choosing an explicit, named feature set over a learned representation is therefore not a limitation forced by compute but a decision that follows directly from the research question, and it is what lets Chapter 6 report not just that style discriminates but how and in which direction.

4.2 System architecture

The artefact is organised as a linear, modular pipeline in which each stage has a single responsibility and a well-defined interface to the next: a data-loading and splitting module, a feature-extraction module, a training module, an evaluation module, and an interpretation module, coordinated by a single orchestration entry point (Figure 4.1). This separation makes the system easy to test, to reason about, and to extend - adding a SHAP-based interpretation or an additional model affects only one module. Deliberately, the feature-extraction module depends only on the standard library and numpy, so the scientific core runs in any environment and can be unit-tested without the heavier machine-learning dependencies.



Figure 4.1 - Pipeline architecture. Each stage has a single responsibility and a fixed interface, and every stochastic step is seeded for reproducibility.

4.3 Feature-set specification

The feature set is the central design artefact and is specified to be interpretable as well as discriminative, so the eventual importance ranking is meaningful. It comprises 95 features in six groups, summarised in Table 4.1: 25 document-level stylometric and lexical features plus the relative frequencies of 70 English function words. The inclusion of each group is justified by the literature: function words and vocabulary-richness statistics are the classical authorship-attribution signal (Mosteller and Wallace, 1964; Stamatatos, 2009), while regularity of length and readability operationalise the expectation that machine prose is more uniform and fluent than human writing.

Feature group	Count	Examples	Rationale
Volume & length	7	char/word/sentence counts; mean & s.d. of word and sentence length	Machine output tends to be more uniform in length
Vocabulary richness	5	type–token ratio, hapax & dis-legomena ratios, Honoré's R, Yule's K	Classical authorship signal; lexical diversity differs by author
Readability	3	Flesch Reading Ease, Flesch–Kincaid grade, syllables/word	Fluency and register of machine prose
Punctuation & orthography	7	comma, period, exclamation, question, uppercase, digit ratios	Stylistic habits and formatting regularity
Structural / phishing cues	3	URL count, email-address count, urgency-word ratio	Interpretable content markers (tested for artefact in §6.4)
Function-word frequencies	70	the, a, of, to, you, is, were, do, our ...	Unconscious, topic-independent stylometric fingerprint

Table 4.1 - The 95-feature stylometric and lexical feature set (see `src/features.py` for exact definitions).

4.4 Requirements prioritisation (MoSCoW)

To manage an intensive timeline the requirements were prioritised. The Must-have elements are the corpus, the feature extractor, the Random Forest and XGBoost models, and the core metric report. The Should-have element is a fine-tuned transformer baseline. The Could-have elements are SHAP-based interpretation and a cross-generator robustness test. The Won't-have elements, explicitly out of scope, are any live deployment, multi-language support, and non-credential-harvesting attack types. This prioritisation guaranteed that even under time pressure the artefact would still answer the research question, because the Must-have set alone is sufficient - and, as Chapter 5 records, the transformer baseline was ultimately cut in line with the proposal's contingency plan without weakening the answer.

5. Implementation

This chapter reviews the key problems encountered while building and running the artefact and how each was overcome, rather than narrating every line of code; the full source is referenced in Appendix A. Five problems were substantive enough to shape the result: corpus construction and class comparability, train/test leakage, feature-extraction edge cases, hyperparameter search under modest compute, and the decision to cut the transformer baseline. A review of the testing process closes the chapter.

5.1 Corpus construction and class comparability

The most consequential engineering was not modelling but data. The two source corpora arrive in incompatible shapes: the AI data are 865 individual plain-text files each headed by a synthetic subject line, whereas the real-world data are large mbox archives of raw MIME messages with headers, HTML parts and quoted-printable encodings. A naive load would have handed the classifier trivial, non-stylistic giveaways - MIME boundaries, HTML tags and header keywords appear in one class and never in the other. The corpus builder therefore parses each mbox message with the standard-library email parser, prefers the text/plain part and falls back to HTML stripped to text, strips the synthetic 'Header:' line from the AI files so both classes are body-only, normalises whitespace and Unicode, and applies a stopword-ratio heuristic to drop non-English bodies.

A rule-based credential-harvesting screen is then applied to both classes so the corpus matches the project's stated scope, addressing the risk that the AI source in particular contains event invitations and other non-credential messages. The operational definition is fixed in the build script: a message is retained only when it pairs an account-, credential- or verification-related subject with an action request - a visible URL, link or button text, form or reply instruction - matched after Unicode normalisation, invisible-character folding, Cyrillic-homoglyph normalisation and stemming. The rule is deterministic and applied identically to both sources, but determinism is not correctness, so it was manually audited (Section 5.2). The screen excluded 306 of the 865 AI messages (retaining 559) and 167 real-world messages, from 1,069 raw Nazario messages of which 652 passed after cleaning. Exact-duplicate removal on lower-cased alphanumeric text then removed reformatted copies of repeated templates, and a seeded down-sample balanced the classes to 491 emails each (982 in total). Restricting the real-world class to 2022–2024 also reduces, without eliminating, temporal mismatch with the 2024 AI class. Every decision and count lives in the runnable build script.

5.2 Validating the credential-harvesting screen

Because the screen determines which messages enter the corpus, its validity underpins the claim that the study concerns credential harvesting specifically, so it was audited rather than assumed correct. A seeded stratified random sample of 100 messages - 25 retained and 25 excluded from each source - was drawn and manually classified against the operational definition. The audit was conducted by the author alone; no second rater was available, so no inter-rater agreement statistic is reported. The individual judgements are recorded in the artefact so they can be re-examined.

The first round exposed a genuine defect. The action condition was originally met only by an explicit verb ('click', 'visit') or a visible URL; every one of the 865 AI messages contains a literal URL, but only about two-fifths of the real-world messages do, because converting an HTML email to text discards the anchor's href and leaves only its label ('UPDATE MY DETAILS'). The rule was thus near-trivially satisfied for one class and structurally unsatisfiable for much of the other, and roughly 60% of the sampled excluded real-world messages proved to be genuine credential lures - a source-dependent selection effect that would have biased the real-world class towards messages that happened to spell out their call to action. The screen was corrected by adding an imperative call-to-action arm covering button text, folding invisible characters and Cyrillic homoglyphs (used to break up keywords) before matching, and matching on word stems.

The corrected rule was rescored on the same 100-message audit sample used to identify the defect; it was not evaluated on a fresh independent audit set, so the following estimates are exploratory and may be optimistic. Among retained messages, 20 of 25 Nazario messages and 23 of 25 AI messages were judged in scope, giving retention precision of 0.80 (Wilson 95% CI [0.61, 0.91]) and 0.92 ([0.75, 0.98]). Among excluded messages, 5 of 25 Nazario messages and 0 of 25 AI messages were judged in scope, giving false-negative rates of 0.20 ([0.09, 0.39]) and 0.00 ([0.00, 0.13]). Weighting the strata by their population sizes gives estimated recall of 0.94 and 1.00; no confidence interval is claimed for these extrapolated estimates. Residual false positives are document-signature and payment-advice lures that mention an account term; residual false negatives are unusually phrased mailbox-recovery notices. The final rule is therefore useful but imperfect and remains marginally stricter on the real-world class. An independent second audit is required before any deployment claim.

5.3 Preventing train/test leakage

With repeated templates in the data, the risk of the same email appearing in both training and test was acute and would inflate the metrics silently. The primary guard is exact-duplicate removal before any split: a message is hashed on its lower-cased alphanumeric characters, so copies differing

only in whitespace, case or punctuation are collapsed and cannot straddle the partitions. Being an exact-after-normalisation match, this does not catch near-identical variants that differ in wording; that residual risk is quantified by the similarity-cluster grouped split in Section 6.5. The split is stratified by class under a single fixed seed (42), yielding 686 training, 148 validation and 148 test emails (74 per class in the test partition). The validation partition was created to provide early-stopping data for the transformer baseline; because that baseline was cut under the proposal's contingency (Section 5.5), it is not used in any reported analysis, and hyperparameters are selected by five-fold cross-validation inside the training partition only. The test partition was excluded from feature fitting, model training and hyperparameter selection, and was used only for final evaluation and the stated sensitivity analyses; the whole run is deterministic.

5.4 Feature-extraction edge cases

Extracting 95 numeric features from arbitrary email text surfaced a series of edge cases that each required a defensive decision. Empty or near-empty bodies produce zero-denominator ratios, so every rate is computed through a safe-division helper that returns zero rather than raising. Vocabulary-richness statistics such as Honoré's R are undefined when every word is unique or the text is a single token, so guarded formulae fall back to zero in those degenerate cases. Syllable counting for the readability indices uses a vowel-group approximation, which is accurate enough for a relative comparison between classes without importing a pronunciation dictionary. Keeping the extractor dependent only on the standard library and numpy was a deliberate constraint: it let the same code be unit-tested in isolation and guarantees that every feature can be explained in plain terms.

5.5 Hyperparameter search under modest compute

The Random Forest and XGBoost models are tuned by five-fold cross-validated grid search on the training data only, optimising F1. The grids were kept deliberately compact - a few hundred candidate fits rather than thousands - because the experiment had to run on a laptop, and because the stylometric signal, as it turned out, is strong enough that the models are not sensitive to fine tuning. The selected configurations were a 400-tree forest of depth None and an XGBoost model of depth 6 with 600 rounds. A late obstacle was environmental rather than statistical: XGBoost failed to load on macOS until the OpenMP runtime it links against was installed - an environmental dependency issue that justifies pinning dependencies in requirements.txt.

5.6 Cutting the transformer baseline

The proposal listed a fine-tuned transformer as a Should-have comparison baseline and named it as the first item to cut if compute ran short. That contingency was exercised. Once the stylometric models were returning very high F1 scores with tight confidence intervals, a transformer baseline would have added considerable compute and complexity without changing the answer to the research question, which concerns stylometric features specifically. Cutting it was a scope decision consistent with the proposal's stated contingency plan, and its implications are discussed in Chapter 7.

5.7 Testing and sanity baselines

The scientific core is covered by unit tests exercising the feature extractor and the metric functions, including the degenerate inputs above; all tests pass. Beyond unit testing, the design guards against the classic failure mode of reporting an impressive number that is really an artefact. A neutral synthetic smoke-test dataset exists purely to prove the pipeline runs end-to-end and is never reported as a result. More importantly, the very high headline numbers prompted a deliberate scepticism that is addressed quantitatively in Sections 6.4 and 6.5 through a structural-cue ablation and a similarity-cluster grouped split, so the evaluation tests whether the result is genuine rather than merely asserting it.

6. Evaluation

6.1 Headline results

All three classifiers were evaluated on the held-out test set of 148 emails, which was excluded from feature fitting, model training and hyperparameter selection. Every model exceeded the $F1 = 0.90$ target pre-specified in the proposal by a wide margin (Table 6.1). The logistic-regression baseline achieved $F1 = 0.987$ (95% bootstrap CI [0.965, 1.000], ROC-AUC 0.995), XGBoost achieved $F1 = 0.980$, and Random Forest achieved $F1 = 0.973$. That a linear model performs at least as well as the tree ensembles indicates that the two classes are close to linearly separable in this feature space. Scores this high on one partition should nonetheless be interpreted cautiously because they may reflect source-specific differences or repeated templates. Sections 6.4 and 6.5 therefore provide complementary stress tests. The similarity-cluster result in Table 6.4 is corroborative, but it is not a controlled performance comparison because it uses a larger training set.

Model	Precision	Recall	F1	F1 95% CI	ROC-AUC	Accuracy
Logistic regression	0.974	1.000	0.987	[0.965, 1.000]	0.995	0.986
Random Forest	0.986	0.959	0.973	[0.942, 0.994]	0.999	0.973
XGBoost	0.986	0.973	0.980	[0.953, 1.000]	0.998	0.980

Table 6.1 - Random-split test-set performance for the positive (AI-generated) class. Interpret alongside the similarity-cluster split in Table 6.4.

6.2 Confusion matrices

The confusion matrices confirm that errors are few and of both kinds. The logistic-regression model misclassified 2 of the 148 test emails, flagging 2 real-world messages as AI-generated while missing none of the AI-generated messages (Figure 6.1); the Random Forest traded recall for precision, missing 3 AI-generated messages and raising 1 false alarm. For a defensive filter that trade-off is the operative choice: the operating point can be shifted along the ROC curve to favour precision or recall as deployment requires.

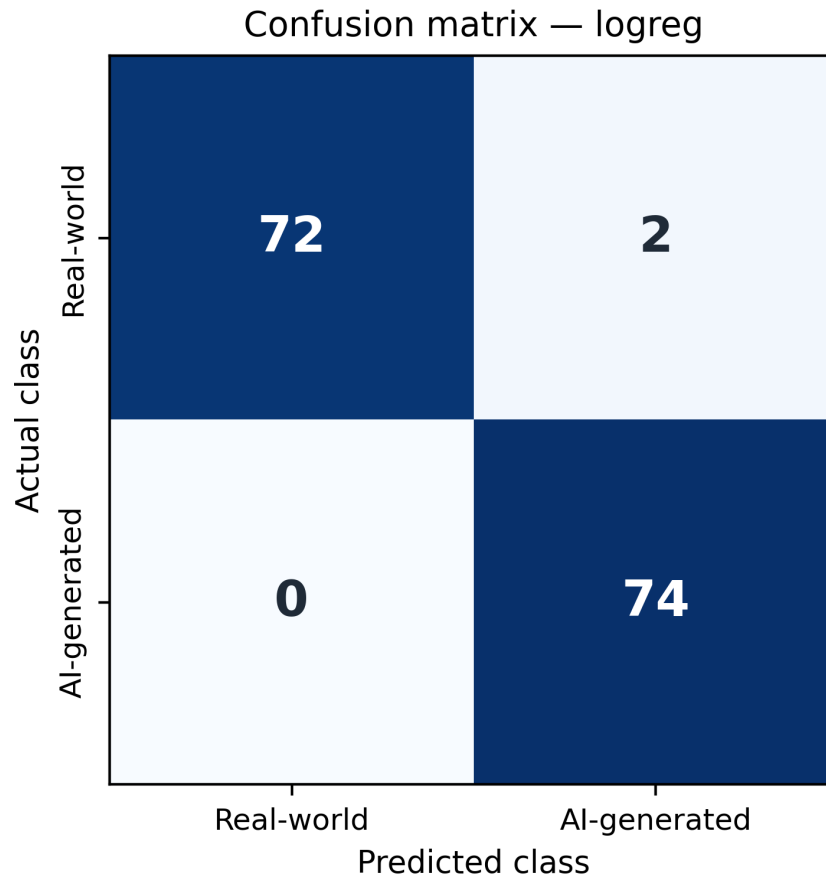


Figure 6.1 - Confusion matrix for the logistic-regression model on the held-out test set. Rows are the true class, columns the prediction.

6.3 Which stylistic markers reveal machine authorship

The second half of the aim asks which markers carry the signal. Two importance methods were computed on the Random Forest - impurity-based and permutation-based - and per-class means (Table 6.2) establish the direction of prominent class differences. The clearest descriptive separator is readability: AI-generated emails have a mean Flesch Reading Ease of 28.1 against 60.0 for real-world phishing, and a correspondingly higher Flesch–Kincaid grade (12.8 versus 7.4). The AI corpus also has longer words on average (5.82 versus 4.86 characters) and a higher urgency-word ratio (0.093 versus 0.055), whereas the real-world corpus contains more digits (0.030 versus 0.007) and heavier terminal punctuation. These are corpus-level differences rather than proof of a universal AI style. The model-based rankings must be interpreted alongside their disagreement, discussed below.

Feature	Real-world mean	AI mean	Reading of the signal
Flesch Reading Ease	60.0	28.1	AI text is harder to read
Flesch–Kincaid grade	7.4	12.8	AI writes at a higher grade
Avg. word length	4.86	5.82	AI uses longer words
Urgency-word ratio	0.055	0.093	AI leans on urgency vocabulary
Digit ratio	0.030	0.007	Real-world phishing carries more digits
Word count	107	86	AI messages are shorter

Table 6.2 - Direction of the strongest signals: per-class feature means over the full corpus.

The two importance methods must be reported together rather than merged, because they do not agree closely, and saying so is more honest than presenting a single ranking as settled. Impurity importance is dominated by the readability and word-length cluster (average word length, Flesch–Kincaid grade, Flesch Reading Ease, syllables per word), whereas permutation importance promotes function-word frequencies and structural counts (the relative frequency of 'the', email-address count, 'please', Honoré's R). Only 7 features appear in both top-20 lists, and across those the Spearman rank correlation is 0.00 ($p = 1.00$) - no reliable agreement on ordering. The divergence is explicable: impurity importance is computed on training data and favours continuous, high-cardinality features such as the readability indices, whereas permutation importance measures the test-set cost of destroying a feature and so rewards what the fitted model actually relies on. Together they support a weaker but defensible claim - that both a readability and length cluster and a function-word cluster carry discriminative information - rather than one definitive ranking (Table 6.3).

Rank	Impurity-based (RF)	Permutation-based (RF)
1	avg_word_length	fw_the
2	flesch_kincaid_grade	email_addr_count
3	flesch_reading_ease	fw_please
4	avg_syllables_per_word	honore_r
5	digit_ratio	fw_you
6	period_ratio	dis_legomena_ratio
7	email_addr_count	avg_word_length
8	std_word_length	sentence_count

Table 6.3 - Top eight features by each importance method. The two methods share only 7 features across their full top-20 lists (Spearman $\rho = 0.00$), so both are reported rather than combined.

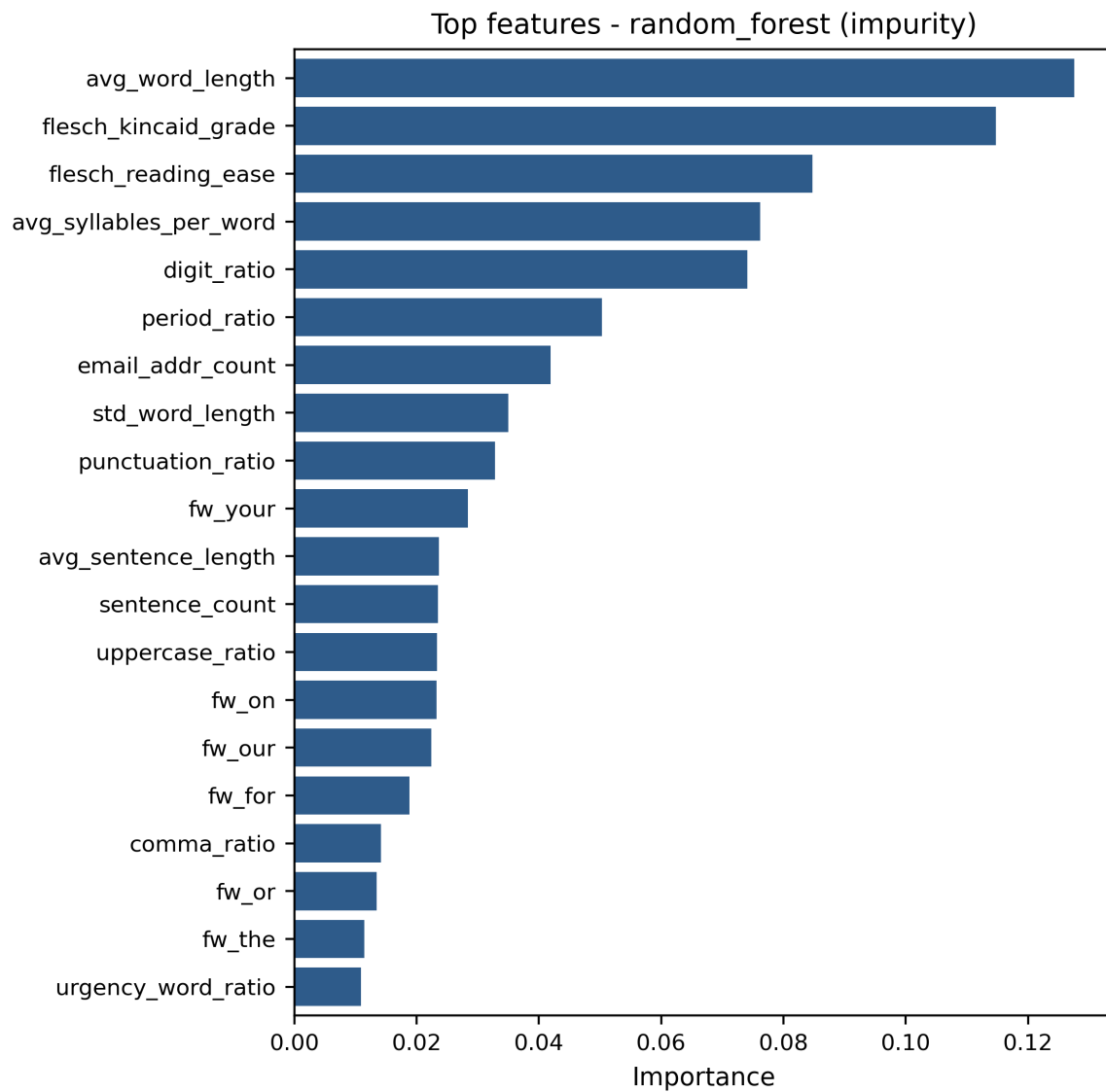


Figure 6.2 - Random-Forest impurity importance (top 20 features).

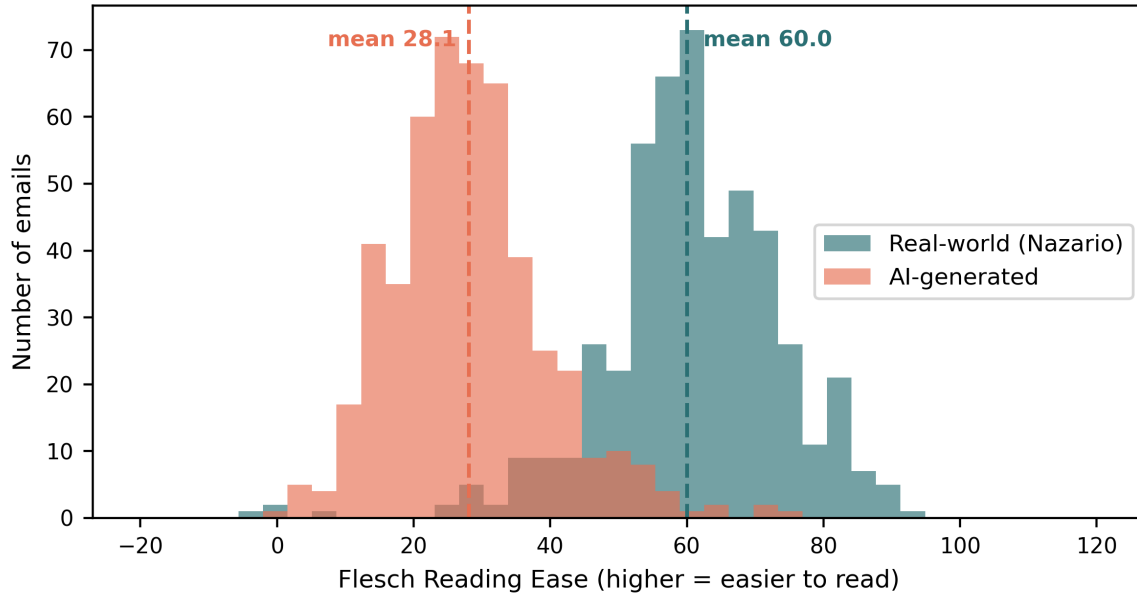


Figure 6.3 - Distribution of Flesch Reading Ease by class. The real-world distribution sits markedly higher (easier to read); dashed lines mark the class means.

6.4 Structural-cue ablation

One pair of features demanded scrutiny before the result could be trusted. The email-address count is almost perfectly separating - real-world phishing from the Nazario mailboxes contains real addresses (mean 1.10 per message) while the AI corpus, generated without real recipients, contains essentially none (0.00) - and URL count differs similarly. These are properties of how each corpus was collected rather than of writing style. To test their influence both were removed and the models retrained. Performance changed little: logistic-regression F1 moved from 0.987 to 0.974, XGBoost from 0.980 to 0.973 and the Random Forest from 0.973 to 0.973, with overlapping confidence intervals in every case. This indicates the result is not driven solely by those two obvious structural cues. It does not, however, establish the absence of all corpus-source confounding: differences in collection process, prompt-template regularities, greetings and sign-offs, length or topic distributions could still contribute, and would require cross-source or matched-topic evaluation to exclude fully.

6.5 Similarity-cluster grouped split and 2022-only sensitivity

Because phishing campaigns often reuse templates with small edits, a random split can place near-identical variants in both training and test sets. To bound this risk, all 982 emails were clustered by textual similarity using character 5-gram shingles and Jaccard similarity of at least 0.5, yielding 547 detected clusters (430 singletons; largest cluster 24 emails). The split was redrawn so that no messages belonging to the same detected similarity cluster could straddle training and test. On the

resulting 833/149 partition (74 AI-generated and 75 real-world test messages), logistic regression and XGBoost each achieved F1 = 1.000, while Random Forest achieved F1 = 0.993 (Table 6.4). These results show that performance remains high when detected template clusters are separated. They must not be read as a direct improvement over the random split, however: the grouped run used 833 training messages, whereas the random-split run trained on 686 messages and reserved 148 for the unused validation partition. Because both grouping and training-set size changed, the absence of a performance drop cannot by itself rule out all template leakage. A matched-size, repeated group-aware cross-validation design would provide a stronger estimate.

Evaluation	Test n	Pos.	Logistic F1	Random Forest F1	XGBoost F1
Random split (Table 6.1)	148	74	0.987 [0.965, 1.000]	0.973 [0.942, 0.994]	0.980 [0.953, 1.000]
Structural-cue ablation	148	74	0.974 [0.943, 0.994]	0.973 [0.942, 0.994]	0.973 [0.942, 0.994]
Similarity-cluster grouped split	149	74	1.000 [1.000, 1.000]	0.993 [0.978, 1.000]	1.000 [1.000, 1.000]
2022-only subset (86/class)	26	13	0.963 [0.870, 1.000]	0.963 [0.870, 1.000]	0.963 [0.870, 1.000]

Table 6.4 - F1 with 95% bootstrap intervals under robustness and sensitivity conditions, for all three classifiers. The signal persists under similarity-cluster splitting and on the 2022-only sensitivity subset.

One caveat applies to Table 6.4. Where a model classified every test message correctly the bootstrap interval collapses to [1.000, 1.000]; that is an artefact of resampling an error-free prediction set, not a claim that generalisation is known exactly. The interval captures sampling variation in the test predictions alone, not uncertainty from model fitting, corpus construction or choice of partition, which would require repeated group-aware cross-validation to quantify.

A second sensitivity analysis addresses the authorship uncertainty of the real-world class. Because the 2023–24 archives overlap heavy public LLM use, the real-world class was restricted to the 2022 archive - the least likely to contain AI-assisted text - and rebalanced (86 per class, 26 held-out test messages). The signal survived (logistic F1 0.963, Random Forest 0.963, XGBoost 0.963), though the small sample gives wide confidence intervals (Table 6.4). Two caveats apply. This cannot prove human authorship; it shows only that the result does not depend on the most exposure-prone years. And restricting the real-world class to 2022 while the AI class remains a 2024 dataset increases the

temporal mismatch between the classes, so this is an authorship-contamination sensitivity check rather than an independent confirmation free of chronological confounding.

6.6 Revisiting the aim, and what the evaluation cannot show

The aim was to determine the extent to which stylometric features distinguish known AI-generated from real-world phishing and to identify which features contribute to the separation. Both halves are addressed: F1 remains between 0.963 and 1.000 across the random split, structural-cue ablation, similarity-cluster partition and 2022-only sensitivity test. Class means show strong readability and word-length differences, while importance analyses indicate contributions from both readability/length and function-word features; their exact ordering is method-dependent. The F1 = 0.90 target was met in every reported condition. Three boundaries remain. The AI class comes from one published generation dataset, so the results do not establish detectability of AI phishing in general. The real-world class is not verified as human-authored. The grouped and random partitions use different training sizes, and neither the static evaluation nor the screening audit models an adaptive adversary or provides independent cross-source validation. High held-out performance is therefore necessary but not sufficient for a deployable detector.

7. Discussion

7.1 Does writing style betray machine authorship?

The direct answer to the research question is yes, within the scope studied. With no access to the generating model, classifiers over 95 interpretable stylometric features separate known AI-generated from real-world phishing with F1 values of 0.973–0.987 on the random split, 0.993–1.000 on the similarity-cluster partition, and 0.963 on the 2022-only sensitivity test. The grouped and random scores are not directly comparable because the grouped run used more training data. The defensible conclusion is therefore that a strong, interpretable signal exists for these two corpora and remains high when detected template clusters are separated - not that perfect general detection has been achieved. Descriptively, the AI corpus is more formal, lexically heavier and more uniform than the real-world corpus, but the exact feature hierarchy is method-dependent.

7.2 Relationship to the literature

The finding replicates and extends Eze and Shamir (2024), who used the stylistic UDAT method to separate AI-generated phishing from human email and ranked several informative descriptors; this project reproduces the core result on their AI corpus against a real-world phishing class, narrows it to a single attack type, and adds ranked interpretation, uncertainty estimates and leakage controls. It is also consistent with the classical stylometry of Mosteller and Wallace (1964) and Stamatatos (2009): the same premise that unconscious stylistic habits betray authorship transfers from the human-versus-human setting to human-versus-machine. At the same time the work inherits, rather than escapes, the central caution of the machine-text-detection literature. Mitchell et al. (2023) and especially Krishna et al. (2023) show that stylistic and statistical detectors are brittle under paraphrasing; there is no reason to think a stylometric detector is immune, and the readability signal that carries most of the discrimination here is exactly the kind of surface property a 'humanising' paraphrase would target. The honest reading is that stylometry is a strong signal against un-adapted LLM phishing and an unproven one against an adaptive adversary.

7.3 Does the artefact solve the stated problem?

The problem defined in Chapter 1 was the detection of machine-authored phishing through writing style. Against that problem, the artefact demonstrates a strong, interpretable separation between the two selected corpora; the result is not driven solely by URL and email-address counts and remains high when detected similarity clusters are separated. The grouped analysis is corroborative rather than a matched comparison, the screen audit is exploratory, and broader source confounding

remains possible. The artefact therefore answers the scoped research question but does not deliver a production-ready detector robust across generators, languages or adaptive attackers. Its contribution is a reproducible characterisation of a precisely bounded signal and a pipeline that future work can test under stronger controls.

8. Conclusion

This study found that a transparent set of stylometric and lexical features distinguished known AI-generated messages from real-world Nazario phishing. On the random held-out split, F1 ranged from 0.973 to 0.987; on the similarity-cluster partition it ranged from 0.993 to 1.000, although the latter used more training data and is not a controlled comparison. Removing URL and email-address counts produced little loss in performance, showing that those two structural cues were not solely responsible for the result. Class-level means reveal pronounced readability and word-length differences, while impurity and permutation importance indicate that both readability/length and function-word features contribute; the exact ranking is method-dependent. The signal also persisted in the 2022-only sensitivity subset. Nevertheless, the findings remain specific to two source corpora and one published AI-generation dataset. Human authorship is unverified, the screening audit was not independently held out, broader source confounding and template leakage beyond the detected clusters remain possible, and cross-generator and adversarial robustness are untested. Stylometry should therefore be viewed as a transparent signal that does not require generator access at inference and may complement, rather than replace, existing phishing controls.

The scope that makes the answer defensible also bounds it, and several lines of future work follow directly. The most important is cross-generator robustness: repeating the study with phishing from several models and services would test whether the readability signature generalises or is specific to this dataset. A second is stronger confound control - matched-topic or matched-length sampling, removal of greetings and organisation names, and evaluation on a fully independent corpus. A third is adversarial robustness - measuring how far deliberate paraphrasing or 'humanisation' degrades the detector, following Krishna et al. (2023). A fourth is extension beyond English-language credential-harvesting email. Addressing these would move the work from a well-characterised result on two corpora towards a detector fit for the adaptive, multilingual threat that AI-generated phishing is becoming.

References

- Abu-Nimeh, S., Nappa, D., Wang, X. and Nair, S. (2007) 'A comparison of machine learning techniques for phishing detection', *Proceedings of the Anti-Phishing Working Group eCrime Researchers Summit*, pp. 60-69.
- Anti-Phishing Working Group (2025) *Phishing Activity Trends Report*. Cambridge, MA: Anti-Phishing Working Group. Available at: <https://apwg.org/trendsreports/> (Accessed: 25 July 2026).
- Bergholz, A., De Beer, J., Glahn, S., Moens, M.-F., Paass, G. and Strobel, S. (2010) 'New filtering approaches for phishing email', *Journal of Computer Security*, 18(1), pp. 7-35.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D. et al. (2020) 'Language models are few-shot learners', *Advances in Neural Information Processing Systems* (NeurIPS) 33, pp. 1877-1901.
- Computer Misuse Act 1990*, c. 18. London: HMSO.
- Das, A., Baki, S., El Aassal, A., Verma, R. and Dunbar, A. (2020) 'SoK: a comprehensive reexamination of phishing research from the security perspective', *IEEE Communications Surveys & Tutorials*, 22(1), pp. 671-708.
- Data Protection Act 2018*, c. 12. London: The Stationery Office.
- Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019) 'BERT: pre-training of deep bidirectional transformers for language understanding', *Proceedings of NAACL-HLT 2019*, pp. 4171-4186.
- Eze, C.S. and Shamir, L. (2024) 'Analysis and prevention of AI-based phishing email attacks', *Electronics*, 13(10), 1839.
- Fette, I., Sadeh, N. and Tomasic, A. (2007) 'Learning to detect phishing emails', *Proceedings of the 16th International Conference on World Wide Web* (WWW '07), pp. 649-656.
- Gehrmann, S., Strobelt, H. and Rush, A.M. (2019) 'GLTR: statistical detection and visualization of generated text', *Proceedings of ACL 2019: System Demonstrations*, pp. 111-116.
- Heiding, F., Schneier, B., Vishwanath, A., Bernstein, J. and Park, P.S. (2024) 'Devising and detecting phishing emails using large language models', *IEEE Access*, 12, pp. 42131-42146.
- International Organization for Standardization (2022) *ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection - Information security management systems - Requirements*. Geneva: ISO.

- Juola, P. (2006) 'Authorship attribution', *Foundations and Trends in Information Retrieval*, 1(3), pp. 233-334.
- Kirchenbauer, J., Geiping, J., Wen, Y., Katz, J., Miers, I. and Goldstein, T. (2023) 'A watermark for large language models', *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR 202, pp. 17061-17084.
- Koide, T., Fukushi, N., Nakano, H. and Chiba, D. (2024) *ChatSpamDetector: Leveraging Large Language Models for Effective Phishing Email Detection*. arXiv:2402.18093.
- Koppel, M., Schler, J. and Argamon, S. (2009) 'Computational methods in authorship attribution', *Journal of the American Society for Information Science and Technology*, 60(1), pp. 9-26.
- Krishna, K., Song, Y., Karpinska, M., Wieting, J. and Iyyer, M. (2023) 'Paraphrasing evades detectors of AI-generated text, but retrieval is an effective defense', *Advances in Neural Information Processing Systems (NeurIPS)* 36.
- Mitchell, E., Lee, Y., Khazatsky, A., Manning, C.D. and Finn, C. (2023) 'DetectGPT: zero-shot machine-generated text detection using probability curvature', *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR 202, pp. 24950-24962.
- Mosteller, F. and Wallace, D.L. (1964) *Inference and Disputed Authorship: The Federalist*. Reading, MA: Addison-Wesley.
- National Cyber Security Centre (2024) *Phishing attacks: defending your organisation*. London: NCSC. Available at: <https://www.ncsc.gov.uk/guidance/phishing> (Accessed: 25 July 2026).
- Nazario, J. (2005-2025) *Phishing Corpus* [dataset]. Licence: CC BY 4.0. Available at: <https://monkey.org/~jose/phishing/> (Accessed: 22 July 2026).
- Sahingoz, O.K., Buber, E., Demir, O. and Diri, B. (2019) 'Machine learning based phishing detection from URLs', *Expert Systems with Applications*, 117, pp. 345-357.
- Solaiman, I., Brundage, M., Clark, J., Askeel, A., Herbert-Voss, A., Wu, J., Radford, A. et al. (2019) *Release Strategies and the Social Impacts of Language Models*. arXiv:1908.09203.
- Stamatatos, E. (2009) 'A survey of modern authorship attribution methods', *Journal of the American Society for Information Science and Technology*, 60(3), pp. 538-556.
- UK General Data Protection Regulation (2021). Retained Regulation (EU) 2016/679.
- Verizon (2025) *2025 Data Breach Investigations Report*. New York: Verizon Business. Available at: <https://www.verizon.com/business/resources/reports/dbir/> (Accessed: 25 July 2026).

Zellers, R., Holtzman, A., Rashkin, H., Bisk, Y., Farhadi, A., Roesner, F. and Choi, Y. (2019) 'Defending against neural fake news', *Advances in Neural Information Processing Systems (NeurIPS)* 32.

Appendix A: Artefact and source code

The complete artefact - feature extractor, data loader, training, evaluation and interpretation modules, unit tests, and the corpus-construction and analysis scripts that produced every table and figure in this report - is version-controlled in the MS-Artifacts/ folder of the repository at [GitHub repository](#). The exact state used for this report is release v1.3-validated, commit da94fdb. The full experiment is reproduced from the raw sources by running, from the MS-Artifacts directory and under the default seed of 42:

```
python3 -m venv .env and pip install -r requirements.txt - environment setup
python scripts/build_corpus.py - rebuild data/emails.csv from the raw sources
python -m src.pipeline --data data/emails.csv --out results/ - train and evaluate
python scripts/supplementary_analysis.py - class means, importance and ablation
python scripts/grouped_split_eval.py - similarity-cluster split and 2022 sensitivity
python scripts/audit_screen_sample.py and audit_screen_score.py - screening audit
python scripts/make_figures.py - architecture and readability figures
python scripts/validate_final_alignment.py - verify the document against the outputs
python scripts/finalise_validated_docx.py and scripts/repaginate_toc.py - assemble this document and refresh its
    contents pages
```

Raw phishing text is excluded from the public repository for the ethical reasons set out in Section 3.6; only aggregate statistics, derived features and dataset checksums are published. The corpus-construction script documents exactly how to rebuild data/emails.csv from the two cited public sources, and the checksums in Table B.3 allow a reader to confirm they obtained identical raw inputs.

Appendix B: Corpus construction and provenance

The tables below record the corpus construction and source provenance as produced by the build script, for full reproducibility. The credential-harvesting screen and its exclusion counts are reported so the scope is evidenced rather than asserted.

Stage	Real-world (Nazario)	AI (Eze & Shamir)
Raw messages parsed	1069	865
Folder-internal pseudo-messages removed	1	0
Removed by length filter	93	0
Removed as non-English	156	0
Excluded by credential-harvesting screen	167	306
Retained before duplicate removal	652	559
Exact duplicates removed	90	68
Remaining after de-duplication	562	491
Removed by balancing down-sample	71	0
Final class size	491	491

Table B.1 - Corpus construction and screening funnel; every stage is reported so the counts reconcile to the final 982-message corpus.

Item	Value
Real-world (class 0) source	Nazario phishing corpus 2022-2024 (CC BY 4.0)
AI (class 1) source	Eze and Shamir (2024), 865 emails via the DeepAI text-generation service
Length filter (chars)	150-8000
Credential screen	Account/credential/verification subject AND an action request via visible URL, link/button text, form or reply, after Unicode, invisible-character, Cyrillic-homoglyph and stem normalisation
Exploratory retained precision (real-world / AI)	0.80 [0.61, 0.91] / 0.92 [0.75, 0.98] (Wilson 95% CI; 25 retained audited per source)
Exploratory estimated recall (real-world / AI)	0.94 / 1.00 (population-weighted; same development audit sample, no independent holdout)
Download date	2026-07-22
Random seed	42

Table B.2 - Corpus parameters, source provenance and exploratory screen-audit performance.

Source file	SHA-256 (first 32 hex chars)
phishing-2022	a6291e2cb990204766fce425423949b4
phishing-2023	5007f1c1dc91b15e93e988afa3a1cf40
phishing-2024	60afa40a2757170093a2bde31ec9cda4
AI_phishing_emails.tar.gz	505245a02b8e02593cec25f51c5ddd32

Table B.3 - Provenance checksums of the raw source files (full hashes in data/corpus_summary.json).

Appendix C: Source code and pseudo-code

The assessment brief asks for pseudo-code in the report and actual code samples in an appendix. The extracts below are taken verbatim from the artefact and cover the decisions that most affect the results: which messages enter the corpus, how each message is turned into features, how campaign-level leakage is controlled, and how uncertainty is estimated. The complete source is in the repository cited in Appendix A.

C.1 Pseudo-code: the credential-harvesting screen

The screen decides which messages enter the corpus, so it is stated here in full. The two conditions are evaluated against obfuscation-normalised text, so that invisible characters and homoglyph substitutions cannot defeat the keyword match (Section 5.2).

```
function IS_CREDENTIAL_HARVESTING(body):
  text <- lowercase(body)
  text <- remove soft hyphens and zero-width characters from text
  text <- map Cyrillic/Greek homoglyphs in text to Latin equivalents

  subject_match <- any ACCOUNT_TERM occurs in text
    # account, password, credential, verific, suspend, expir,
    # billing, mailbox, identity, unusual activity, ...

  action_match <- any ACTION_PATTERN occurs in text      # 'click', 'visit'
    or any CTA_PATTERN occurs in text                    # 'verif', 'renew'
    or text contains a URL                               # http, www, hxxp

  return subject_match AND action_match
```

C.2 Pseudo-code: campaign-level grouped partition

Random splitting can place near-identical campaign variants in both partitions. Clustering by textual similarity and splitting whole clusters bounds that risk (Section 6.5).

```
function GROUPED_SPLIT(corpus, threshold = 0.5, test_fraction = 0.15):
  for each message m in corpus:
    shingles[m] <- set of character 5-grams of alphanumeric(m)
```



```

# union-find over messages that share at least one shingle
for each pair (i, j) sharing a shingle:
    if |shingles[i] AND shingles[j]| / |shingles[i] OR shingles[j]| >= threshold:
        union(i, j)

clusters <- connected components of the union-find structure
for each class c in {real-world, AI-generated}:
    shuffle clusters of class c with the fixed seed
    assign whole clusters to TEST until test_fraction is reached
    assign every remaining cluster to TRAIN

return TRAIN, TEST      # no cluster appears in both partitions

```

C.3 Feature extraction (src/features.py, extract)

```

def extract_features(text: str) -> Dict[str, float]:
    """Return an ordered dict of interpretable features for one email body."""
    words      = _tokenize_words(text)
    sentences  = _split_sentences(text)
    counts     = Counter(words)
    hapax     = sum(1 for w, c in counts.items() if c == 1)

    # readability (Flesch Reading Ease and Flesch-Kincaid grade)
    syllables = sum(_count_syllables(w) for w in words)
    asl = _safe_div(len(words), len(sentences))    # avg sentence length
    asw = _safe_div(syllables, len(words))         # avg syllables per word
    feats['flesch_reading_ease'] = 206.835 - 1.015 * asl - 84.6 * asw
    feats['flesch_kincaid_grade'] = 0.39 * asl + 11.8 * asw - 15.59

    # vocabulary richness
    feats['type_token_ratio'] = _safe_div(len(set(words)), len(words))
    feats['hapax_ratio']     = _safe_div(hapax, len(words))

    # stylometric fingerprint: relative frequency of 70 function words
    for fw in FUNCTION_WORDS:
        feats[f'fw_{fw}'] = _safe_div(counts[fw], len(words))
    return feats

```

C.4 Credential-harvesting screen (scripts/build_corpus.py, extract)

```

def _match_text(text: str) -> str:
    """Normalise away obfuscation so the screen sees the intended words."""
    return _INVISIBLE_RE.sub('', text).translate(_HOMOGLYPHS).lower()

def is_credential_harvesting(text: str) -> bool:
    """Two-condition screen: account/credential subject AND a solicited action."""
    low = _match_text(text)
    has_subject = any(t in low for t in ACCOUNT_TERMS)
    has_action = (
        any(t in low for t in ACTION_PATTERNS)
        or any(t in low for t in CTA_PATTERNS)
        or bool(_URL_HINT_RE.search(low))
    )
    return has_subject and has_action

```

C.5 Bootstrap confidence interval (src/metrics.py, extract)

```

def bootstrap_f1_ci(y_true, y_pred, n_boot=2000, alpha=0.05, seed=42):
    """Percentile bootstrap CI for F1, by resampling test predictions."""
    rng = np.random.default_rng(seed)
    n = len(y_true)
    stats = []
    for _ in range(n_boot):
        idx = rng.integers(0, n, n)          # sample with replacement
        stats.append(f1_score(y_true[idx], y_pred[idx], zero_division=0))

```

```
lo = float(np.percentile(stats, 100 * alpha / 2))
hi = float(np.percentile(stats, 100 * (1 - alpha / 2)))
return {'ci_low': lo, 'ci_high': hi}
```